Known Issues

All known release issues related to this project will be documented in this file.

> *If you have a known issue that is not listed here, please open an issue on the project's GitHub repository.*

Table of Contents

- 1.0 Confluent Control Center (C3) never becomes Ready when it wins the race against Kafka's DNS
  - 1.1 Symptom
  - 1.2 Cause
  - 1.3 Workaround
- 2.0 Every image pull fails with `TLS handshake timeout` (`ErrImagePull` / `ImagePullBackOff`)
  - 2.1 Symptom
  - 2.2 Cause
  - 2.3 Workaround

1.0 Confluent Control Center (C3) never becomes Ready when it wins the race against Kafka's DNS

Affects: `cp-enterprise-control-center-next-gen:2.5.0` on minikube (`make cp-core-up`)

1.1 Symptom

`make c3-open` reports a healthy port-forward, but the browser shows `ERR_CONNECTION_REFUSED` on http://localhost:9021. `kubectl get pods -n confluent` shows the pod stuck at **2/3 Running with 0 restarts** — it never crash-loops and never recovers on its own:

```
NAME              READY   STATUS    RESTARTS   AGE
controlcenter-0   2/3     Running   0          45m
```

The readiness probe fails continuously:

```
Readiness probe failed: Get "http://10.244.0.7:9021/2.0/status/app_info":
connect: connection refused
```

1.2 Cause

All Confluent Platform pods are applied at once, so Control Center can start before the `kafka` headless Service has endpoints. C3 resolves `bootstrap.servers` once, eagerly, with no retry — when DNS is not

ready yet, its `main` thread dies during Guice provisioning:

```
WARN  [main] Couldn't resolve server kafka:9071 from bootstrap.servers as
DNS resolution failed for kafka
Exception in thread "main" com.google.inject.ProvisionException:
  Caused by: KafkaException: Failed to create new KafkaAdminClient
  Caused by: ConfigException: No resolvable bootstrap urls given in
bootstrap.servers
```

Non-daemon Kafka Streams threads keep the JVM alive after `main` exits, so the container never terminates, the kubelet never restarts it, and it never binds port `9021`. The pod stays in this state indefinitely.

Two details make this hard to spot:

- The **fatal exception is in the middle of the log, not at the tail** — `kubectl logs --tail` shows only `Unable to obtain lock as state directory is already locked by another process`, which is a downstream symptom of the dead `main` thread, not the cause. Grep for `No resolvable bootstrap urls` to confirm.
- `kubectl port-forward` attaches to any *Running* pod regardless of *readiness*, so the port-forward itself genuinely succeeds while nothing is listening on the other end.

1.3 Workaround

Restart the pod once Kafka is up; the StatefulSet recreates it and it reaches 3/3 in about a minute:

```
kubectl delete pod controlcenter-0 -n confluent
kubectl wait --for=condition=Ready pod/controlcenter-0 -n confluent --
timeout=240s
make c3-open
```

`make c3-open` is gated on `make c3-ready`, which waits for the pod to report Ready (up to `C3_READY_TIMEOUT`, default `180s`) and prints this diagnosis instead of opening a browser onto a dead port.

2.0 Every image pull fails with `TLS handshake timeout` (`ErrImagePull` / `ImagePullBackOff`)

Affects: minikube on a VM driver (`vfkit` on macOS, `kvm2`/`qemu` on Linux) when the host reaches the internet but traffic forwarded from the VM does not, typically because of a third-party network extension, content filter, or VPN on the host

2.1 Symptom

The first pod to start — usually `confluent-operator` during `make cp-up` — never pulls its image, and `make cp-watch` cycles between `ErrImagePull` and `ImagePullBackOff`:

```
NAME                                  READY   STATUS             RESTARTS
AGE
confluent-operator-6dbff485d6-kwq6n   0/1     ImagePullBackOff   0
2m37s
```

`kubectl describe pod` shows containerd timing out against Docker Hub itself, not a missing tag:

```
Failed to pull image "docker.io/confluentinc/confluent-operator:0.1718.99":
failed to pull and unpack image
  "docker.io/confluentinc/confluent-operator:0.1718.99": failed to resolve
reference ...:
  Head "https://registry-1.docker.io/v2/confluentinc/confluent-
operator/manifests/0.1718.99": net/http: TLS handshake timeout
```

Yet the same registry answers fine from the host itself. Every other image in the stack (Kafka, Schema Registry, Control Center, cp-flink, RustFS, aws-cli) fails the same way, and `make flink-image-build` fails on its base-image pull and the Containerfile's `curl` fetch stage.

2.2 Cause

Something on the host drops the reply traffic for connections forwarded from the VM, while connections the host opens itself work. Every site fails from inside the node, yet ping and the TCP connect still succeed, so it looks like a registry outage rather than a local routing problem. The usual culprit is a third-party network extension or content filter; check System Settings → Network → Filters & Proxies, and `systemextensionsctl list`.

Quick confirmation — direct egress from the node times out, while the same request through an HTTP proxy on the host (see §2.3) returns `401`, Docker Hub's normal unauthenticated answer:

```
minikube ssh -- "curl -s -o /dev/null -w '%{http_code}\n' --max-time 10
https://registry-1.docker.io/v2/"
# 000
minikube ssh -- "curl -s -o /dev/null -w '%{http_code}\n' --max-time 10 -x
http://192.168.64.1:3128 https://registry-1.docker.io/v2/"
# 401
```

2.3 Workaround

If you can't remove the extension or filter, run any HTTP proxy that supports `CONNECT` on the host (for example tinyproxy or squid). It must listen on an address the VM can reach and allow the VM's subnet. The proxy then opens the outbound connections as a host process, which works. For example, with tinyproxy on macOS:

```
brew install tinyproxy
# In $(brew --prefix)/etc/tinyproxy/tinyproxy.conf: set 'Port 3128' and add
'Allow 192.168.64.0/24'
```

Then set `MINIKUBE_HTTP_PROXY` for this machine in a git-ignored `local.mk` (start from `local.mk.example`):

```
MINIKUBE_HTTP_PROXY ?= http://192.168.64.1:3128   # the Mac as seen from
the vfkit VM
```

From then on, every target uses the proxy:

- `make minikube-start` starts the tinyproxy Homebrew service and writes `HTTP_PROXY` / `HTTPS_PROXY` / `NO_PROXY` systemd drop-ins for containerd and BuildKit in the node. The node's `/etc` is tmpfs, so this is redone on every start.
- `make flink-image-build` passes the same variables as build args, which covers the Containerfile's `curl` fetch stage.
- `make minikube-stop` and `make minikube-delete` stop the service again (set `MINIKUBE_PROXY_BREW_SERVICE` empty to manage it yourself).

For a node that's already running, use `make minikube-proxy-apply`; pods in `ImagePullBackOff` recover on their next retry. To go direct for a single run, use `MINIKUBE_HTTP_PROXY= make …`. `MINIKUBE_NO_PROXY` keeps cluster-internal traffic off the proxy.